

AXON-RFC-006 附录 B: easynet.web_app

由 agent 部署的全栈应用：技术设计、构建管线、Hub 路由、actions.* 命名空间与实现计划

胡思蓝
新加坡国立大学

草稿 v0.2, April 28, 2026

Abstract

本附录规定 `easynet.web_app`：用于支撑完整现代 web 部署的 `state_type`。一个 agent (典型情形为 Claude Code) 在 `~/.easynet/pages-project/<slug>/` 项目目录中撰写真实的 React/Vue/Next 风格前端项目，通过构建产生静态 `dist/`，该 `dist/` 与一个由 EasyNet ability 直接充当的后端一起被服务。Hub 扮演有状态版的 `nginx` 角色：服务前端 bundle、把 `POST /api/<verb>` 翻译成 owner agent 上的 ability 调用、把 WebSocket 订阅代理到 ability stream，并通过 `/__easynet/agent` 暴露结构化的 `agent_view`。除 `serve-and-translate` 之外，本附录还引入 `actions.*` ability 命名空间——可查询的接口 (`actions.list_for_page`、`actions.get`、`actions.search`)，peer agent 通过它按需取得与某个具体页面或意图相关的 EAL 片段与 mission，而不必批量下载静态清单。Agent skill 自动种子 (`easynet-pages-author`) 在守护进程首次接触 agent 时安装到其 workspace，让“撰写 `web_app`”成为一项可学习的技能而不是隐式协议知识。本附录覆盖：项目位置与生命周期；状态机 (canonical + runtime 叠加)；长任务构建管线；Hub 路由矩阵；`actions.*` 契约与 `/easynet-for-agents/<verb>` HTTP 镜像；`agent_view` 在新设计下的角色；在 EasyNet-CLI 中落地所需的五个 PR 切片；以及一个端到端的 worked example——agent 用 Vite + React 写购物前端、声明 API 与 `actions` 面、构建、发布、上线、被审计的全过程。

Contents

1	位置与脉络	2
1.1	为什么这不是 <code>nginx</code>	3
1.2	相对 v0.1 的变化	3
2	目标与非目标	3
2.1	目标	3
2.2	非目标	4
3	项目模型	4
3.1	项目位置	4
3.2	<code>easynet.app.toml</code>	5
3.3	为什么 API 与 <code>actions</code> 面必须声明而不是自动检测	6
3.4	Agent skill 默认 seed	6
4	状态机	7
4.1	<code>state_type</code> 与 <code>state_key</code>	7
4.2	规范字段	7
4.3	规范状态空间	8
4.4	Runtime 叠加状态空间	8
4.5	Transition 列表	8
5	构建管线	8

5.1	webapp.build@v1 步骤	8
5.2	TreeBlob 格式	9
5.3	构建执行器	9
6	actions.* ability 命名空间	9
6.1	为什么是命名空间，不是静态目录	10
6.2	推荐 verb 集	10
6.3	Action 描述符	10
6.4	EAL 实际是什么（以及不是什么）	11
6.5	例：actions.list_for_page + actions.get	11
6.6	Hub 镜像语义	12
6.7	协议不强制的事	12
7	Hub 路由模型	13
7.1	按状态划分的 Hub 行为矩阵	13
7.2	Action 分派	13
8	agent_view	14
9	架构	15
9.1	模块拓扑 (EasyNet-Cli)	15
9.2	守护进程接线	16
9.3	Hub 接线	16
10	实现计划	16
10.1	P-B1: 构建执行器 + TreeBlob	16
10.2	P-B2: webapp.* 能力	17
10.3	P-B3: Hub 静态 + API + actions 路由	17
10.4	P-B4: WebSocket 代理 + SPA fallback	17
10.5	P-B5: agent_view + meta 端点 + skill seed	17
11	端到端 worked example	18
11.1	准备	18
11.2	编辑项目	18
11.3	构建与上线	18
11.4	浏览器拉取	18
11.5	浏览器侧 API 调用	19
11.6	邻居 agent 发现 actions	19
11.7	审计重放	20
12	暂缓至 RFC-006-B v2 的开放问题	20
12.1	覆盖检查	21

1 位置与脉络

[推论] web 从一个静态文档媒介演变成完整应用平台；本附录沿同一弧线，作为 RFC-006 两个附录中的第二个：

附录	建模对象
A (easynet.page)	单一文档。作者写 HTML / Markdown; Hub 服务它。最简参考——单一规范状态、无 runtime 叠加、无构建步骤、无 API 面。
B (easynet.web_app)	完整应用。作者写真实前端项目, 构建管线产生静态 bundle, 应用的 API endpoint 即 EasyNet ability, 且 peer agent 可经 actions.* ability 命名空间询问 owner ”在这一页上能做什么”。

两者都保留。Pages 仍是协议规范核最简化的验证; web_app 是第一个调用长任务规范 transition、runtime 叠加、完整 Hub 路由、以及 agent 可执行接口发现的状态类型。

1.1 为什么这不是 nginx

nginx 只服务一个静态目录、并把请求反向代理到独立部署的后端。后端对 nginx 是黑盒——它不理解后端状态、不知道后端有哪些 endpoint, 也无法对一个通过结构化发现机制询问”调用方在这一页上能做什么”的邻居作答。本附录规定的 Hub 服务的字节与 nginx 一样, 但其路由决策派生自 state object 的规范状态、其 API endpoint 是和其他每个 EasyNet 调用方共享同一注册表的、可内省的 ability, 并且其发现表面 (actions.*) 不是侧带 manifest 而是 owner 拥有的可查询一等命名空间。nginx 是 HTTP 管道; Hub 是恰好在外边界讲 HTTP、在内边界讲 EAL 的状态投影层。

1.2 相对 v0.1 的变化

自上一稿起的三处结构性改动:

1. **项目位置**。项目目录从 <workspace>/web_apps/<slug>/ 改到独立、全局的 ~/.easynet/pages-project/<slug>/。理由: 项目属于一个 state object, 其 owner 已经在 state_key 中编码, 不必再通过目录嵌套表达 ownership。
2. **Agent skill 默认 seeded**。一个名为 easynet-pages-author 的捆绑 skill 在守护进程首次接触 agent 时安装到其 workspace, 与现有的 easynet-ability-crud 同模式。该 skill 教 agent 如何 scaffold、声明、构建、上线、发现 web_app。撰写一份 web_app 是可学习的技能, 不是口耳相传的知识。
3. **actions 是可查询的 ability 命名空间**。上一稿曾把”actions_view”设计成与页面并列的、放着 EAL 文件的静态目录。那种设计强迫 peer agent 不论与当前页/当前意图是否相关都批量下载所有 action——这正是 peer 想消费 agent 可执行接口表面时最不愿遇到的事。新设计把 actions.* 引入为 EasyNet ability 命名空间: peer agent 调用 <owner>.actions.list_for_page(page_path) 即按需得到 仅与该页面相关的 actions。/easynet-for-agents/<verb> HTTP 镜像存在以便浏览器侧使用, 但协议级契约是 ability, 不是 URL。

2 目标与非目标

2.1 目标

[规范性] 一个符合本附录的实现:

1. 让 agent 在 ~/.easynet/pages-project/<slug>/ 中用任意工具链 (Vite、Next.js、Remix 等) 撰写真实前端项目; EasyNet 不发明项目结构。
2. 提供长任务规范 transition webapp.build, 运行项目声明的构建命令、把产出的 dist/ 以一份不可变的内容寻址 bundle 形式捕获, 并按成功/失败推进 canonical state。

3. 提供 runtime 叠加 transition (`webapp.serve`、`webapp.stop`) 绑定 Hub 映射；canonical state 不变。
4. 在 Hub 上把 HTTP 请求按三条正交通道翻译为 EasyNet 操作：静态 bundle 拉取、API ability 调用 (`/api/<verb>`)、WebSocket ability 订阅 (`/ws/<topic>`)。
5. 在 `/__easynet/agent` 暴露 `agent_view` 作为应用状态与可发现性元数据的类型化 JSON 文档，含指向 `actions.*` 命名空间的指针。
6. 定义 `actions.* ability` 命名空间 (§6)：一个由 owner 实现、可查询的接口表面，peer agent 通过它按需请求 EAL 片段与 mission。HTTP 镜像 `/easynet-for-agents/<verb>` 是 ability 调用的别名，不是静态目录。
7. 在每个 agent 的 workspace 中 seed `easynet-pages-author skill`，让”撰写 `web_app`”成为 agent 自身有据可查的能力 (§3.4)。
8. 可在五个 PR 切片 (§10) 中落地，每个切片可独立审阅。

2.2 非目标

[推论] 不在本附录范围：

- TLS 终结、HTTP/2 协商、多实例 Hub 集群——属于运维。
- 选定的前端框架——本附录使用 React/Vite 仅因 Claude Code 默认这套；协议本身与框架无关。
- 持续进程 SSR——Phase 1 是静态 + Hub 路由 API + WebSocket；每请求都需要 Node.js 进程响应的 SSR 推迟到 RFC-006-B v2。
- 跨 realm 或跨节点应用联邦——Phase 1 单 owner / 单节点。
- 已认证人类 (`principal/human-auth`) ——Phase 1 仅 `principal/human-anon`。
- 把”三视图等价”作为严格协议不变量——希望 `human_view` 与 `actions` 内容承载完全相同的信息确实有价值，但 v1 无法以协议形式编码；`actions.*` 命名空间是 owner 自愿声明的 agent 可执行表面，不是从其他视图派生的。

3 项目模型

3.1 项目位置

[规范性] state object 在磁盘上的真值来源是位于以下路径的项目目录：

```
~/.easynet/pages-project/<slug>/
easynet.app.toml           # EasyNet 侧的声明
package.json               # agent 工具链所有
package-lock.json          # 同上
src/                       # 框架风格源码
public/
vite.config.ts              # 或 next.config.js 等
(可选) 框架需要的任何其他文件
```

EasyNet 只拥有 `easynet.app.toml`；其余属于框架。该位置全局（每主机一棵目录树，按 slug 寻址）而非每 agent (`<workspace>/...`)，因为 `state_key` 已经携带 owner 身份——把目录嵌套到 agent workspace 下会重复编码 ownership 并使 Hub 访问复杂化（Hub 通过 blob store 读字节，而调试运维不应该需要先知道某 slug 归哪个 agent）。

目录在 `webapp.create` 时由守护进程创建；agent 通过自己的工具链（`npm create vite` 等）填入框架文件。守护进程在创建之后不向该目录写入，除非通过该 `state object` 的 `transition`（如 `webapp.delete` 时清空）。位于 `~/.easynet/blobs/` 的 `blob` 存储是 Hub 服务字节的规范来源；项目目录是 agent 的工作副本。

不变量 — WB-INV-1

`webapp.build` 成功之后，Hub **必须** 仅从 `blob` 存储以 `build_artifact_hash` 为键服务字节。项目目录的 `dist/` 可以为 agent 调试方便保留在磁盘上，但 Hub **绝不** 读取它用于服务。为服务而读取项目的 `dist/` 是非合规。

3.2 easynet.app.toml

[规范性] EasyNet 用以理解 `web_app` 的唯一文件：

```
# easynet.app.toml — EasyNet 编写并阅读的唯一文件。
# 项目其余部分属于框架。

[manifest]
name          = "shop"
description    = "由 agent 构建的小型陶瓷店。"
visibility     = "public"           # private | realm | public

[build]
command        = "npm install && npm run build"
dist_dir       = "dist"             # 相对项目根
build_timeout_secs = 600
node_version   = "20"              # 建议性

[serve]
mode           = "static-only"
               # "static-only" : 无后端进程；API 完全由 Hub 发起的
               #                  ability 调用满足。Phase 1 默认与
               #                  生产形态。
               # "process"      : 启动后端进程；预留给 v2 (SSR) 。

[api]
# 前端\emph{允许}通过 Hub 的 /api/<verb> 路由调用的 ability。Hub 会
# 拒绝把 verb 不在此列表的 /api/<verb> 请求翻译。
abilities = [
  "shop.list_items",
  "shop.checkout",
  "shop.order_status",
]

[actions]
# 经 /easynet-for-agents/<verb> 镜像暴露的、由 owner 实现的
# `actions.*` ability。每条是 owner agent 已注册的 actions.*
# 命名空间下的 verb。peer agent 调用这些以按需发现 EAL 片段与
# mission。详见 §6。
abilities = [
  "actions.list_for_page",
  "actions.get",
  "actions.search",
]

[routes]
```

```
# nginx 风格路由规则。顺序重要。
rules = [
  { match = "exact:/",                action = "static:dist/index.html" },
  { match = "prefix:/assets/",        action = "static:dist/assets/" },
  { match = "regex:~/api/([a-z][a-z0-9_]*)$",
    action = "api:$1",
    methods = ["GET", "POST"] },
  { match = "regex:~/easynet-for-agents/([a-z][a-z0-9_]*)$",
    action = "actions:$1",
    methods = ["GET", "POST"] },
  { match = "prefix:/ws/",            action = "ws:$path" },
  { match = "exact:/__easynet/agent", action = "agent_view" },
  { match = "exact:/__easynet/health", action = "health" },
  { match = "exact:/__easynet/receipts", action = "receipts" },
  { match = "fallback",              action = "spa:dist/index.html" },
]

[csp]
extra_script_src = []
extra_style_src = ["'unsafe-inline'"]
extra_connect_src = ["self"]
```

最小应用三个段 ([manifest]、[build]、[api]) 即可; [actions] 可选 (§6 解释为何); routes 默认走上面 SPA + actions 标准布局。

3.3 为什么 API 与 actions 面必须声明而不是自动检测

[规范性] [api] 与 [actions] 两个面都是声明而非发现的。当对应 verb 未声明时, Hub 拒绝把 /api/<verb> 或 /easynet-for-agents/<verb> 翻译为调用。

不变量 — WB-INV-2

当 <verb> 不在 web_app 当前 Built canonical state 的 api_surface_hash (即规范化的 [api].abilities 列表) 中时, Hub **必须**拒绝把 /api/<verb> 翻译为调用。同规则对 /easynet-for-agents/<verb> 与 actions_surface_hash 适用。拒绝必须返 HTTP 404 (不是 403) 以避免确认内部 ability 的存在——身份层 TR-INV-12 建立的信任边界纪律在应用层的实例化。

3.4 Agent skill 默认 seed

[规范性] 守护进程在首次接触每个 agent 时, 把一个名为 easynet-pages-author 的捆绑 skill 安装到该 agent 的 workspace 标准 skill 路径下:

```
<workspace>/skills/easynet-pages-author/SKILL.md
<workspace>/agents/skills/easynet-pages-author/SKILL.md
```

该 skill 形态与现有的 easynet-ability-crud (commit 3bc6c70) 完全一致。它讲清楚:

- 怎样 scaffold 一个前端项目(npm create vite@latest <slug> -- --template react-ts);
- 项目标准位置(~/.easynet/pages-project/<slug>/)——在 webapp.create 铸造 state_key 并创建目录之后使用; agent cd 进守护进程准备好的目录;
- easynet.app.toml 的结构, 含 [api] 与 [actions] 段;
- 通过现有 easynet-ability-crud skill 注册 api.* ability 的方法;

- 怎样实现 actions.* 命名空间 (§6) 以让 peer agent 发现该应用的可执行表面；
- 生命周期: webapp.create → 填项目 → webapp.build → webapp.serve；
- 不需重建即可改路由或 CSP 时如何用 webapp.update_config；
- 故障排查 (构建失败、端口冲突、visibility 拦下的 404)。

该 skill 是 Phase 1 的 P-B5 交付物 (§10)。它不进 canonical state——它在 agent workspace 中、归 agent 所有，仅在守护进程检测到丢失或过时重新种子。

4 状态机

4.1 state_type 与 state_key

[规范性]

```
state_type: easynet.web_app

state_key:
    (realm: string, owner_agent_uri: string, app_slug: string)

URA 形态:
    easynet:///r/<realm>/agent/<owner>/web_app/<slug>
```

app_slug 匹配 Appendix A pages 中定义的 slug ABNF。位于 ~/.easynet/pages-project/<app_slug>/ 的目录在 webapp.create 时创建、webapp.delete 时清除。

4.2 规范字段

[规范性] 按 CSH-INV-2 (主文 §2.5) 大体积值都按内容哈希持有：

```
canonical_fields(web_app) = {
    manifest_hash:      bytes(32), // [manifest] 段
    build_config_hash:  bytes(32), // [build] 段
    api_surface_hash:   bytes(32), // [api].abilities (已排序)
    actions_surface_hash: bytes(32), // [actions].abilities (已排序)
    routes_manifest_hash: bytes(32), // [routes].rules (按声明顺序)
    csp_hash:           bytes(32), // [csp] 段
    source_snapshot_hash: bytes(32), // 上次成功构建源树
                                // 首次构建前为 null
    build_artifact_hash: bytes(32), // dist 树哈希
                                // 首次 Built 前为 null

    state_value:        string,
    visibility:          string,
    version:             uint64,
}
```

每字段在 schema 中标 canonical:true；schema 驱动的 π 投影恰按这些 key 产出。

4.3 规范状态空间

state_value	含义
Created	已注册 easynet.app.toml；项目目录已存在；尚未尝试构建。
Building	webapp.build 在进行中。瞬态。仅经 OperationalReceipt 可见；从不作终态规范状态。
Built	已存在至少一次成功构建。build_artifact_hash 非空。可被 Served。
Failed	最近一次构建失败。owner 可迭代后重试。
Deleted	终态；项目目录与 blob 引用清除。

4.4 Runtime 叠加状态空间

runtime	含义
Stopped	此 app 没有附着的服务叠加。
Starting	webapp.serve 已被调用；Hub 正在把 slug 绑定到其路由。
Running	Hub 正主动为此 app 翻译 /<slug>/... 请求。
Crashed	Hub 丢失了映射（进程重启、配置重载错误等）；canonical state 未触动。可经 webapp.serve 恢复。

4.5 Transition 列表

Transition	类	说明
webapp.create@v1	canonical	$\emptyset \rightarrow \text{Created}$ 。铸造 state_key、创建项目目录、落各段哈希。
webapp.update_config@v1	canonical	Created/Built/Failed $\rightarrow$ 同。替换声明配置(manifest、build、api、actions、routes、csp)。不触发构建。
webapp.build@v1	canonical	Created/Built/Failed $\rightarrow$ Building（瞬态） $\rightarrow$ Built Failed。长任务。见 §5。
webapp.delete@v1	canonical	* $\rightarrow$ Deleted。隐式触发 webapp.stop 副作用；清空项目目录。
webapp.serve@v1	operational	Stopped/Crashed $\rightarrow$ Starting $\rightarrow$ Running。要求 canonical = Built。
webapp.stop@v1	operational	Starting/Running/Crashed $\rightarrow$ Stopped。
webapp.get@v1	query	返回 canonical + runtime 快照。
webapp.list@v1	query	列出某 agent 拥有的 web_app。

5 构建管线

5.1 webapp.build@v1 步骤

[规范性] webapp.build 时执行器：

1. 读取 ~/.easynet/pages-project/<slug>/easynet.app.toml 并校验。

2. 计算 `source_snapshot_hash`: 项目目录的 Merkle 哈希 (排除 `node_modules/`、`dist/`、`.git/` 以及框架 `.gitignore` 排除的任何路径)。
3. 发出 `build.started OperationalReceipt`, 含源快照哈希、声明命令、预期超时。
4. 在项目目录内 `spawn` 构建命令。`stdout/stderr` 三通; 按周期发 `build.progress OperationalReceipt` (可丢失)。
5. 退出时:
 - 状态 0 且 `dist_dir` 非空: 遍历 `dist_dir`; 按 Merkle 计算 `build_artifact_hash`; 把整棵 `dist` 树作为单个 `TreeBlob` (一份按 JCS 规范化的 manifest, 列文件路径与各自 blob 哈希) 写到 blob 存储。
 - 状态非零或 `dist_dir` 空: 构建失败原因 `payload` 并存其哈希。
6. 发出唯一一条 `CanonicalReceipt`: 成功为 `build.completed`, 失败为 `build.failed`。该 `transition` 的唯一规范推进; `post_state_hash` 反映 Built 或 Failed; `state_version + 1`。

瞬态 Building、流式进度、构建工具 `stdout` 都是观测; 它们绝不进入规范哈希。

5.2 TreeBlob 格式

[规范性]

```
TreeBlob (JCS 规范化 JSON)
{
  "version": 1,
  "entries": [
    { "path": "index.html",
      "size": 2148,
      "blob": "<file 字节的 sha256>",
      "mode": "0644" },
    ...
  ]
}
```

树的哈希是 `sha256(jcs(TreeBlob))` —— `build_artifact_hash`。每个独立文件也是内容寻址 blob。

5.3 构建执行器

[规范性] Phase 1 交付一个本地构建执行器: 构建在拥有 `agent` 的 EasyNet 守护进程的子进程中运行。`owner` 直接为产生的 `CanonicalReceipt` 签名。委托构建 (源在 `agent A`、构建在 `agent B`) 属于未来: 协议形态已就绪 (主文 §5.3), 但执行器尚未实现。

待决策

待决策: 构建沙箱。 构建子进程具备守护进程的进程权限。Phase 1 缓解: 构建在限制 `PATH`、不继承除 `NODE_VERSION` 与 `HOME` 外的环境变量、更严的 `ulimit` 下运行。完整沙箱 (`seccomp` / `pledge` / `Docker`) 属于 RFC-006-B v2。

6 actions.* ability 命名空间

6.1 为什么是命名空间，不是静态目录

[推论] peer agent 访问一个 web_app 时，它不想下载 owner 写过的每一个 EAL 片段；它想拿到与当前页或当前意图相关的 *actions*。静态 EAL 文件目录强迫批量下载与客户端过滤——上一稿就卡在这里。重新设计把 agent 可执行表面当成可查询资源。

owner agent 在 *actions.** 命名空间下注册 ability。peer agent (或 Hub 浏览器) 按名调用这些 ability。owner 的实现决定返回什么：按页相关集、按意图匹配集、完整目录、单个具名 action，或任何未来形态。因为 *actions* 是 ability，所有现存协议机制全部适用：ability_surface、schema、scope check、receipt、审计、联邦路由。

6.2 推荐 verb 集

[规范性] 鼓励但不要求 owner 实现一组标准 verb：

Ability	用途
<code>actions.list_for_page(page_path)</code>	返回与 <code>page_path</code> 相关的 <i>actions</i> 。最常用入口：刚访问某页的 agent 问“我在这能做什么”。
<code>actions.get(name)</code>	返回某具名 action 的完整定义——通常是 EAL 或 mission 文本，加结构化的 <i>inputs</i> schema。
<code>actions.search(intent)</code>	返回与自由文本意图字符串匹配的 action 描述符的有序列表。排序由 owner 决定。
<code>actions.list_all()</code>	返回完整目录。等价于 sitemap；爬虫与镜像有用。

verb 集是约定，不是协议。owner 可实现子集、超集，或一个都不实现；agent 调一个不存在的 verb 得到普通的 `MethodNotFound`。Hub 的 `/easynet-for-agents/<verb>` 镜像只转发到 owner 注册的任意 verb。

6.3 Action 描述符

[规范性] 任意 *actions.** verb 返回的描述符形态：

```
{
  "name":      "checkout",           // owner 命名空间
  "kind":      "eal" | "mission" | "form_map",
  "summary":   "下一笔订单。",
  "url":       "/easynet-for-agents/get?name=checkout",
                                     // POST 携带 params 以获取已具体化
                                     // 的 mission
  "params_schema": {                 // peer 传给 actions.get 的 params
                                     // 对象的 schema; owner 把这些
                                     // 作为字面量内嵌到返回的 mission
    "item_id": { "type":"string", "required":true,
                  "doc":"用 actions.search 找 ID" },
    "qty":      { "type":"uint",   "default":1 }
  },
  "result_render": {                 // 可选，为 form_map 用
    "success_template": "订单 {order_id}, 预计 {eta}",
    "error_template":   "抱歉: {error_message}"
  }
}
```

action 主体——EAL 片段、mission，或 form_map JSON——都是 *actions.get* 返回的值。因为主体是 ability 返回值而非磁盘文件，owner 可动态计算（如把当前用户的会话注入到 mission

中) 而不违反任何协议规则。视图确定性 (TR-INV-6) 不适用——actions 是 ability 不是 view。

6.4 EAL 实际是什么 (以及不是什么)

[规范性] actions.get 返回的 mission 主体是该代码库已实现的 EAL (src/eal/) 程序。EAL 是有意为之的窄语言:

- 一个 mission 是 call 步骤的静态 DAG。没有 if、没有 fail、没有表达式求值、没有 return。每个步骤的 args 在 mission 文本写下时就已全部填好。
- 参数语法是 with { key = value, ...} (=, 不是 :)。值是 string / int / float / bool 字面量, 或 prev_step.output 变量引用。
- 绑定: let x = call ...。变量只能被引用为 x.output。
- 步骤选项恰好为 timeout N、retries N、on_failure {abort | skip | retry | continue}、optional。
- 唯一 block 形式是 loop, 含 body {...} verify {...}。

这是 src/eal/lexer.rs 与 src/eal/ast.rs 的实际语法。没有 inputs 子句、没有 \$variable 插值、除了 loop 块外没有任何 mission 内控制流。

对 actions 的含义。 因为 mission 文本是静态的, 需要 caller 提供参数的 action 不能在主体里“接收 inputs”。协议改用 **owner 端具体化**: peer agent 把它要的参数传给 actions.get(name, params); owner agent 把这些参数作为字面量内嵌到返回的 mission 文本中。peer 拿到的是已具体化、可直接喂给 mission run 的 mission。

这保留了 receipt 链。mission 中的每个 call 都是真实的 EasyNet invocation, 经现有 dispatcher 走、产生 CanonicalReceipt; mission 的第一个步骤把 actions.get 的 receipt 引用为它的 causal_context.Scalar, 让审计者能证明“这次 mission 运行源于对 alice 的某个 actions.get 调用”。

6.5 例: actions.list_for_page + actions.get

```
$ easynet ability invoke alice.actions.list_for_page \
  --args '{"page_path":"/cart"}'

[
  { "name":"checkout", "kind":"mission",
    "summary":"购买当前用户购物车里的商品。",
    "url":"/easynet-for-agents/get?name=checkout",
    "params_schema":{                                // peer 把这些传给
      "address":{"type":"string","required":true}, // actions.get
      "qty":    {"type":"uint", "default":1}
    } },
  { "name":"clear_cart", "kind":"eal",
    "summary":"清空购物车。",
    "url":"/easynet-for-agents/get?name=clear_cart" }
]

# Peer 让 owner 用 peer 自己的输入把 checkout mission 具体化。
# Owner 把它们作为字面量内嵌到一个真实的 EAL mission 中
# (没有 $variable, 没有 inputs 块)。
$ easynet ability invoke alice.actions.get \
  --args '{"name":"checkout",
          "params":{"address":"123 Main St","qty":1}}'
```

```
mission "checkout" {
  let cart = call "shop.cart_contents" on "alice" with {} timeout 5

  let order = call "shop.checkout" on "alice" with {
    items  = cart.output,
    address = "123 Main St",
    qty    = 1
  } timeout 30
}
```

peer agent 现在直接运行返回的文本：

```
$ easynet mission run /tmp/checkout.eal --cite-receipt <actions.get-receipt-id>
```

--cite-receipt 标志把 `causal_context.Scalar(prior_invocation_id)` 写入 mission 第一个步骤的 invocation envelope, 让 receipt 日志记录 `actions.get` → `shop.cart_contents` → `shop.checkout` 这条链。peer 没引用也仍能正确运行——只是失去了跨状态对象的因果链接, 让”为何此 mission 现在运行”这件事在审计层无证。

对比上一稿的静态目录设计: 同一个 agent 必须拉 `checkout.eal`、`clear_cart.eal`、`list_items.eal` 加索引文件再过滤——一个意图四次拉取加客户端参数替换, 且没有 `causal_context` 把运行链回到提示该运行的页面。

6.6 Hub 镜像语义

[规范性] 路由 action `actions:<verb>` 把 HTTP 请求

```
GET  /easynet-for-agents/<verb>?<query>
POST /easynet-for-agents/<verb>      {<json>}
```

翻译为 EasyNet ability 调用 `<owner>.actions.<verb>`, 把 query string 或 JSON body 作 args, 与 `api:<verb>` 同。结果作 JSON 返回 (或当结果是字符串、且 owner 标了 `Content-Type: text/eal` 时, 原文按该 type 转发)。任何不在 `actions_surface_hash` 的 verb 按 WB-INV-2 拒绝。

说明

HTTP 镜像存在是便利, 不是权威。peer EasyNet agent 可等价地直接经总线调 `<owner>.actions.<verb>`。两条路径产生同种 `OperationalReceipt` 与 `caller-context` 记录。镜像主要是为了浏览器侧脚本可在 `actions` 表面之上构 UI (如”我在这能做什么”下拉), 无需 EasyNet 客户端。

6.7 协议不强制的事

[推论] 三件刻意省略的强制:

- **human_view 与 actions 不要求基数对应。** 页面显示”12 件商品在售”的 owner 不被协议强制让 `actions.list_for_page("/")` 返回 12 件。视图间诚实是 *owner* 纪律; 协议提供机制 (两视图都由同一 owner 签名; 审计链可检测漂移) 但不要求等价作不变量——v1 的强制窗口太窄。
- **无标准 EAL/mission 文本格式版本。** `actions.get` 主体是 owner 自由返回的; EAL 语法在自己的 RFC 中演化。Action 描述符携带 `kind` 让消费者按主体解释分派。

- **无权威排序或相关性。** `actions.list_for_page` 与 `actions.search` 返回 `owner` 定义的排序。需要独立排序的 `peer` 自己实现。

7 Hub 路由模型

Hub 绑定单端口 (Phase 1 默认 127.0.0.1:8787)。每个进入的 HTTP 请求按七步处理：

1. 铸造 envelope: `caller = principal/human-anon/<ip-hash>/<sid>`; `caller_context` 按主文 §3.6 从 header 填。
2. 解析 path 到 (`realm`, `owner`, `slug`, `tail`)。
3. 解析 state。若不在 Built, 按 §7.1 返; 若 visibility 拒绝 caller, 返 404。
4. 按声明顺序在 `routes_manifest` 规则中匹配 tail。
5. 按命中 action 分派: `static / api / actions / ws / agent_view / health / receipts / spa` fallback (§7.2)。
6. 构响应 (CSP、ETag、缓存头、content-type)。
7. 发出一条 `OperationalReceipt`, 含 `caller`、`caller_context`、命中规则、action、响应状态。非规范。

7.1 按状态划分的 Hub 行为矩阵

Canonical	Runtime	Hub 响应
Created	*	503 "未构建"。Retry-After: 60。
Building	*	503 + 通过 SSE 流式发送 <code>build.progress</code> 的小进度页。
Built	Stopped	503 "未在服务"。
Built	Starting	503 "启动中", Retry-After: 2。
Built	Running	完整 action 分派 (下文)。
Built	Crashed	503 + reason hash; Retry-After: 5。
Failed	*	503 "上次构建失败" + reason hash。
Deleted	*	404。

7.2 Action 分派

static:<path> TreeBlob path lookup、blob 拉取、扩展名 MIME (允许列表), 由 `csp_hash` 构 CSP; 对 `/assets/*` 用 `immutable` 缓存。

api:<verb> 按 `api_surface_hash` 校验、把 JSON body 或 query string 作 args、调用 `<owner>.<verb>`、返回 result body 或结构化错误。拒绝: HTTP 404 (WB-INV-2)。

actions:<verb> 按 `actions_surface_hash` 校验、把 JSON body 或 query string 作 args、调用 `<owner>.actions.<verb>`、返回 result body。Content-Type 默认 `application/json`; 若结果体声明 Content-Type: `text/eal`, Hub 原文以该 type 转发 (让 peer 经镜像取 EAL 文件时看到正确文本类型)。拒绝: HTTP 404 (WB-INV-2)。

ws:<path> WebSocket 升级; 首条文本帧必须是 { `ability`, `args` } JSON; 绑到 `ability stream`; 转发后续帧。

agent_view 返回 §8 规定的 JSON。

health 返回 canonical 与 runtime 状态。

receipts 返回受 ?from=...&to=... 约束的 CanonicalReceipt 历史。

spa:<fallback-path> 等价于 **static:<fallback-path>** 但带 Cache-Control: no-cache。

8 agent_view

[规范性] agent_view 是关于应用的结构化元数据：状态、版本、历史、应用宣示的接口面（api + actions）。它不是 action 库——actions 在 actions.* 命名空间中、经该命名空间被查询，不经 agent_view。

```
GET /<realm>/<owner>/<slug>/__easynet/agent

200 OK
Content-Type: application/json

{
  "state_type": "easynet.web_app",
  "state_key": { "realm":"default","owner":"alice","slug":"shop" },
  "ura": "easynet:///r/default/agent/alice/web_app/shop",

  "canonical": {
    "state": "Built",
    "manifest": { "name":"shop",
                  "description":"由 agent 构建的小型陶瓷店。",
                  "visibility":"public" },
    "build_artifact_hash":"0x4d28ee17...",
    "version": 7,
    "etag": "TSjuFw0pAvjpAQ=="
  },

  "runtime": {
    "state": "Running",
    "started_at":"2026-04-28T09:11:02Z"
  },

  "ability_surface": [
    "webapp.update_config@v1",
    "webapp.build@v1",
    "webapp.stop@v1",
    "webapp.delete@v1"
  ],

  "api": {
    "endpoints": [
      { "verb":"shop.list_items",
        "http":"GET /api/list_items",
        "schema_url":"/__easynet/schema/shop.list_items" },
      { "verb":"shop.checkout",
        "http":"POST /api/checkout",
        "schema_url":"/__easynet/schema/shop.checkout" }
    ]
  },
}
```

```

"actions": {
  "namespace": "actions",
  "registered_verbs": [
    "actions.list_for_page",
    "actions.get",
    "actions.search"
  ],
  "http_mirror_prefix": "/easynet-for-agents/",
  "discovery_hint":
    "POST /easynet-for-agents/list_for_page {\"page_path\": \"<path>\"}"
},

"history": {
  "versions": 7,
  "latest_canonical_receipt_id": "r_c3d4...",
  "receipt_log_ura":
    "easynet:///r/default/agent/alice/web_app/shop/receipts"
}
}

```

为什么 `agent_view` 不内嵌 `action` 库。内嵌 `actions` 会迫使 `agent_view` 列出 `owner` 写过的每个 EAL 片段——正是促使引入 `actions` 命名空间的批量下载低效。`agent_view` 指向命名空间；`peer agent` 再向命名空间按需调用。两文档互补：`agent_view` 答”这是什么应用”，`actions` 答”我现在在它上能做什么”。

9 架构

9.1 模块拓扑 (EasyNet-Cli)

<code>src/runtime/state/</code>	[与 <code>pages</code> 共享, P-A1]
<code>src/runtime/blob_store/</code> <code>tree.rs</code>	[与 <code>pages</code> 共享, P-A1] ← 新: <code>TreeBlob</code> <code>put/get</code> 、 <code>per-path</code> lookup
<code>src/runtime/agents/webapp_ability.rs</code>	← 新 (P-B2)
<code>src/runtime/build_executor/</code> <code>mod.rs</code> <code>source_snapshot.rs</code> <code>spawn.rs</code> <code>artifact_capture.rs</code>	← 新 (P-B1) 长任务规范 <code>transition</code> 驱动 项目目录 Merkle 哈希 子进程 + 日志流 dist 树 -> <code>TreeBlob</code>
<code>src/runtime/views/webapp/</code> <code>agent_view.rs</code> <code>health.rs</code> <code>receipts.rs</code>	← 新 (P-B5) §7 的 JSON 投影
<code>src/runtime/hub/</code> <code>router.rs</code>	[扩展 P-A4 的 <code>pages</code> Hub] ← 扩展以读 <code>routes_manifest</code> 与新的 <code>actions</code> : <code>action</code>
<code>static_handler.rs</code>	← 新 (P-B3)
<code>api_handler.rs</code>	← 新 (P-B3) : <code>/api/<verb></code>
<code>actions_handler.rs</code>	← 新 (P-B3) : <code>/easynet-for-agents/<verb></code>
<code>ws_handler.rs</code>	← 新 (P-B4) : <code>/ws/*</code> 升级


```
src/runtime/skills/easynet-pages-author/ ← 新 (P-B5)
SKILL.md                                seeded 到每个 workspace 的
                                         agent 教程
```

9.2 守护进程接线

启动时守护进程在 pages 接线之上加：

1. 在 ~/.easynet/build-cache/ 打开构建执行器工作目录。
2. 给每个 agent 注册 webapp_ability。
3. 对每个规范状态为 Building 的 web_app（即守护进程重启打断了一次构建）：标 Failed 并把 reason 设为”守护进程在构建途中重启”。
4. Seed easynet-pages-author skill 到每个 agent 的 workspace（若不存在或过时）。

9.3 Hub 接线

Hub 二进制 (easynet-hub, P-A4 起) 在 P-B3/P-B4 中加上 static_handler、api_handler、actions_handler、ws_handler，与 P-A4 的 page handler 并列。顶层分派：

```
fn handle(req: HttpRequest) -> HttpResponse {
  let envelope = build_envelope(&req);
  let (realm, owner, slug, tail) = parse_path(&req.uri)?;
  let kind = state_store.classify(realm, owner, slug)?;

  match kind {
    Kind::Page    => pages_dispatch(envelope, owner, slug, tail),
    Kind::WebApp  => webapp_dispatch(envelope, owner, slug, tail),
    Kind::Unknown => http_404(),
  }
}
```

10 实现计划

五个 PR 切片，可顺序化。P-B1 基于 P-A1 (pages 的 blob 与 state 存储)；P-B3/B4 基于 P-A4 (基本 Hub)。

10.1 P-B1：构建执行器 + TreeBlob

新增文件。src/runtime/build_executor/ (4 模块)、src/runtime/blob_store/tree.rs。
公开 API。

```
pub trait BuildExecutor: Send + Sync {
  fn execute(
    &self,
    project_dir: &Path,
    config: &BuildConfig,
    progress: ProgressSink,
  ) -> Result<BuildOutcome>;
}

pub fn source_snapshot_hash(project_dir: &Path, ignore: &[Pattern])
  -> Result<[u8; 32]>;
```

```
pub fn capture_dist(dist_dir: &Path, blob: &dyn BlobStore)
    -> Result<(TreeBlob, [u8; 32])>;
```

测试。快照确定性、构建烟测、守护进程在构建中崩溃恢复。

10.2 P-B2: webapp.* 能力

新增文件。src/runtime/agents/webapp_ability.rs。

行为。§4 中每个 transition 都被实现，包括长任务构建（按 TR-INV-1）。

测试。每个 transition 往返；从 Created 出发的 webapp.serve 被拒；webapp.update_config 上的乐观并发；构建进度回执可丢失。

10.3 P-B3: Hub 静态 + API + actions 路由

新增文件。src/runtime/hub/static_handler.rs、api_handler.rs、actions_handler.rs；扩展 router.rs 以读 routes_manifest。

行为。

1. 静态处理器：TreeBlob path lookup、blob 拉取、对 /assets/* immutable 缓存。
2. API 处理器：捕获 verb、按 api_surface 校验、解析 args、调用 <owner>.<verb>、映射结果。
3. Actions 处理器：捕获 verb、按 actions_surface 校验、解析 args、调用 <owner>.actions.<verb>、映射结果。默认 application/json；尊重 ability 返回的 Content-Type: text/eal。

测试。集成：起一个 fixture web_app，其 api.abilities 含 shop.echo、其 actions.abilities 含 actions.list_for_page；POST /api/echo 回填、POST /easynet-for-agents/list_for_page 返 action 描述符。未授权 verb 返 404。

10.4 P-B4: WebSocket 代理 + SPA fallback

新增文件。src/runtime/hub/ws_handler.rs。

行为。/ws/* 上 WS 升级；首条文本消息必须是 {ability,args} JSON；绑到 ability stream；转发文本帧。SPA fallback：用 no-cache 服务 dist/index.html。

测试。WS 流式烟测、SPA fallback 应对未知路径。

10.5 P-B5: agent_view + meta 端点 + skill seed

新增文件。src/runtime/views/webapp/ (3 模块)；对 /__easynet/* 前缀做轻微路由扩展；捆绑 skill 在 src/runtime/skills/easynet-pages-author/SKILL.md。

行为。

1. /__easynet/agent 返回 §7 的 JSON。
2. /__easynet/health 返 canonical + runtime 状态。
3. /__easynet/receipts 返 CanonicalReceipt 历史。
4. /__easynet/schema/<verb> 返该 ability 的输入/输出 schema。
5. Skill seeder：启动时守护进程把捆绑 easynet-pages-author skill 写到每个 agent 的 workspace <workspace>/skills/ 与 <workspace>/.agents/skills/。幂等：已存在且最新时跳过。

测试。agent_view 字节确定性；skill seeder 幂等性；从回执重放重构 canonical state。

11 端到端 worked example

11.1 准备

```
$ easynet agent add alice --type claude-code
$ easynet-daemon &          # 首次接触 alice 时 seed easynet-pages-author skill
$ easynet-hub --bind 127.0.0.1:8787 &
```

11.2 编辑项目

Claude Code 读完 easynet-pages-author 的 SKILL.md 后，请求守护进程铸造 state 并创建项目目录：

```
$ easynet ability invoke alice.webapp.create --args '{
  "slug":"shop",
  "manifest":{ "name":"shop",
               "description":"由 agent 构建的小型陶瓷店。",
               "visibility":"public" }
}'
{ "state":"Created", "version":1,
  "project_dir":"~/.easynet/pages-project/shop" }
```

agent 然后 cd 进项目目录、用工具链：

```
$ cd ~/.easynet/pages-project/shop
$ npm create vite@latest . -- --template react-ts
$ # ... 写 src/App.tsx, 注册 API 与 actions ability ...
$ # ... 按 §3.2 填 easynet.app.toml ...
$ easynet ability deploy abilities/shop.list_items      --node alice
$ easynet ability deploy abilities/shop.checkout        --node alice
$ easynet ability deploy abilities/actions.list_for_page --node alice
$ easynet ability deploy abilities/actions.get          --node alice
```

11.3 构建与上线

```
$ easynet ability invoke alice.webapp.build --args '{ "slug":"shop" }' \
  --stream
[build.started   ] command="npm install && npm run build"
[build.progress  ] log_hash=0x12... 8% installed
[build.progress  ] log_hash=0x13... 47% bundled
[build.completed] artifact_hash=0x4d28ee17... duration=38.2s
{ "state":"Built", "version":2,
  "build_artifact_hash":"0x4d28ee17..." }

$ easynet ability invoke alice.webapp.serve --args '{ "slug":"shop" }'
{ "runtime_state":"Running" }
```

11.4 浏览器拉取

```
$ curl -i http://127.0.0.1:8787/default/alice/shop/

HTTP/1.1 200 OK
Content-Type: text/html
```

```
ETag: "TSjuFwOpAvjpAQ=="
Cache-Control: max-age=60, must-revalidate
Content-Security-Policy: default-src 'self'; script-src 'self'; ...

<!doctype html>...
```

11.5 浏览器侧 API 调用

```
POST /default/alice/shop/api/checkout HTTP/1.1
Cookie: easynet_session=sess-abc
{ "item_id":"mug-12oz", "qty":1 }

→ HTTP/1.1 200 OK
{ "order_id":"ord_77a3f", "eta":"2026-04-29" }
```

11.6 邻居 agent 发现 actions

邻居 agent (bob) 落到购物车页，问 owner 它能做什么：

```
$ curl -s -X POST http://127.0.0.1:8787/default/alice/shop/easynet-for-agents/
  list_for_page \
    -d '{"page_path":"/cart"}' | jq
[
  { "name":"checkout", "kind":"mission",
    "summary":"购买当前用户购物车里的商品。",
    "url":"/easynet-for-agents/get?name=checkout",
    "params_schema":{
      "address":{"type":"string","required":true}
    } },
  { "name":"clear_cart", "kind":"eal",
    "summary":"清空购物车。",
    "url":"/easynet-for-agents/get?name=clear_cart" }
]

# bob 让 owner 用 bob 选择的地址把 mission 具体化。owner 把字面量
# 内嵌到一个真实的 EAL DAG 中 (没有 $variable —— EAL 不支持; 见 §6.4) 。
$ curl -s -X POST "http://127.0.0.1:8787/default/alice/shop/easynet-for-agents/get" \
  -d '{"name":"checkout",
      "params":{"address":"123 Main St"}}'

mission "checkout" {
  let cart = call "shop.cart_contents" on "alice" with {} timeout 5

  let order = call "shop.checkout" on "alice" with {
    items = cart.output,
    address = "123 Main St"
  } timeout 30
}
```

两次 RPC、精确 action、地址由 owner 内嵌为字面量。bob 现在直接运行返回的文本，引用 actions.get 的 receipt 让运行首步带上跨状态对象的因果链：

```
$ easynet mission run /tmp/checkout.eal --cite-receipt <actions.get-receipt-id>
```

每个 call 经现有 dispatcher 走、产生 CanonicalReceipt；首步的 invocation envelope 携带 causal_context.Scalar 指向 actions.get 的 receipt，让审计者沿着链看到 actions.list_for_page

→ actions.get → shop.cart_contents → shop.checkout 这条连续因果。

11.7 审计重放

shop web_app 的历史显示三条 CanonicalReceipt(create、build、build): 每条都带 post_state_hash 与 owner_signature。浏览器驱动的 checkout 触发了一条针对完全不同 state_type 的 CanonicalReceipt (一个由 alice 或 alice 的订单服务拥有的 order state object); 该回执也链回 alice 的签名密钥。同时阅读两条链的邻居可逐字节重构:

- 当 checkout 发生时, page 处于哪个构建产物;
- 那个构建声明了哪些 API 与 actions endpoint;
- actions.get 给驱动该 checkout 的请求返回了哪段 mission 主体;
- 上述全部由 alice 签名。

这是传统 web 决然讲不出的审计故事。

12 暂缓至 RFC-006-B v2 的开放问题

1. 持续进程 SSR (serve.mode = process)。
2. 跨节点 Hub、联邦读取。
3. 已签资源 bundle 用于 SRI。
4. 构建沙箱。
5. 多版本路由 (webapp.tag transition)。
6. dev-mode 反向代理 (serve.mode = dev-proxy) 以支持 HMR / Vite dev server。

12.1 覆盖检查

主文规则	由谁兑现	位置
SO-INV-1 (key 不可变)	state_key 解析器拒绝 key 更新	§4
CSH-INV-1 (hash 取自 projection)	schema 驱动的 π 、JCS、SHA-256	§4
CSH-INV-2 (内容外置为 blob)	TreeBlob + 单文件 blob; 规范状态只持哈希	§5
RC-INV-1 (receipt 绑定)	每条 webapp 回执均带 (state_key, transition_id, attempt_id)	§4
RC-INV-2 (乐观并发)	在 update_config 上做 ETag 检查	P-B2
TA-INV-1 (必须带 transition_id)	每个 webapp.<verb>@v1 字符串均被校验	§4
VP-INV-1 (ability_surface 来自 state)	由 state_value 派生	§8
TR-INV-1 (长任务终结)	构建只发恰一条终结 CanonicalReceipt	§5
TR-INV-2 (progress 不进规范)	build.progress 是 OperationalReceipt	§5
TR-INV-5 (runtime 叠加独立)	runtime 崩溃不触动 canonical	§4
TR-INV-7 (multi-view 一等公民)	human (HTML+API+WS) 与 agent_view 都已实现	§7, §8
TR-INV-8 (cross-state-space 矩阵)	canonical $\times$ runtime 准入被强制	§4
TR-INV-11 (URA-receipt 可追踪)	/__easynet/receipts + state_key 索引	§11
TR-INV-12 (principal/human-anon)	Hub 在每个请求上铸造该 URI	§7
TR-INV-13 (caller_context)	由请求头填充; 记录到 operational receipt	§7, §11
WB-INV-1 (仅从 blob 服务)	Hub 永不读项目 dist/	§3
WB-INV-2 (api/actions 面强制)	Hub 拒绝未列出 verb 并返 404	§3

结语。 agent-native web 的可发现性问题现在在正确的层得到解决。peer agent 落到一页, 调 actions.list_for_page, 得到的正是与该页相关、用其自身意图参数化的 EAL/mission ——不是要过滤的 sitemap, 不是要下载的 manifest。撰写是可学习技能, 不是口耳相传: 每个 agent 在首次接触时获得 easynet-pages-author。项目位于稳定且与 agent 无关的位置 (~/.easynet/pages-project/), 同一份 codebase 不论被哪个 agent 操作读起来都一致。落地 P-B1 至 P-B5 完成 Phase 1。

web 从文档演变到应用; agent-native web 从被动视图演变到可查询 actions。先 pages、后 web_app; 不论 pages 还是 web_app, actions 都是 peer agent 不批量下载就能使用它们的方式。